

Architettura degli elaboratori e sistemi operativi

Diego Stefanini — prof. Marco Danelutto, a.a. 2026-27

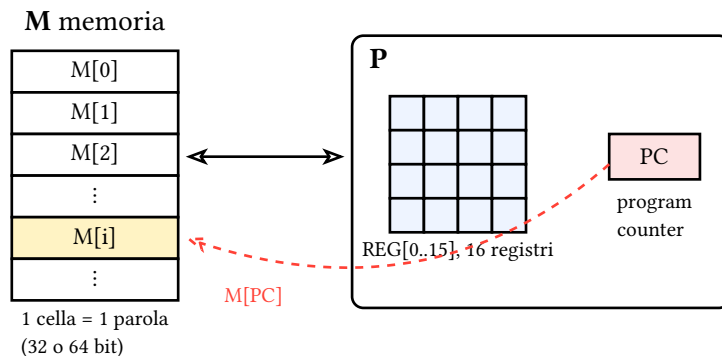
Indice

1	Fondamenti dei sistemi di elaborazione	3
1.1	Memoria, processore, istruzioni	3
1.2	Astrazione	4
1.3	Numeri binari	6
1.4	Basi, numeri reali e caratteri	9
2	Reti logiche	11
2.1	Porte logiche e tabelle di verità	11

1 Fondamenti dei sistemi di elaborazione

1.1 Memoria, processore, istruzioni

Un calcolatore è fatto di due parti che si scambiano dati di continuo: la **memoria** M, che contiene programmi e dati, e il **processore** P, che esegue i programmi.



Il PC contiene l'indirizzo della prossima istruzione da prendere in memoria

- La **memoria** è un lungo vettore di celle numerate. Il numero di una cella è il suo **indirizzo**: $M[i]$ è la cella di indirizzo i . Ogni cella contiene una **parola** di 32 o 64 bit (un bit è una cifra 0 o 1, vedi i numeri binari più avanti).
- I **registri** sono poche celle (qui 16) **dentro** il processore: molto più veloci della memoria, ci stanno i dati su cui il processore sta lavorando in quel momento.
- Il **PC** (*program counter*) è un registro speciale: contiene l'indirizzo della prossima istruzione da eseguire. Anche il programma, infatti, sta in memoria.

Una volta avviato, il processore ripete per sempre questi quattro passi, il **ciclo fetch-decode-execute**:

```
while (true) {  
    preleva l'istruzione da M[PC]  
    decodifica      // che istruzione è?  
    esegue  
    scrive i risultati  
}
```

Dopo il prelievo il PC passa all'istruzione successiva, così al giro dopo il processore prende quella dopo.

Quali istruzioni esistono dipende dal processore (x86, ARM, RISC-V sono famiglie di processori diverse), ma i tipi sono sempre tre:

Tipo	Istruzioni	Esempio
operative	ADD SUB MUL DIV AND OR NOT SHIFT ROT (sx/dx)	ADD R0, R1, R2 → $REG[0] = REG[1] + REG[2]$ SHIFT sposta i bit a sinistra o a destra; ROT li ruota: il bit che esce da una parte rientra dall'altra
memoria	LOAD STORE	LOAD R5, addr → $REG[5] = M[addr]$ (addr = un indirizzo) STORE fa il contrario: registro → memoria
salto	salto	cambia il PC: la prossima istruzione non è quella dopo

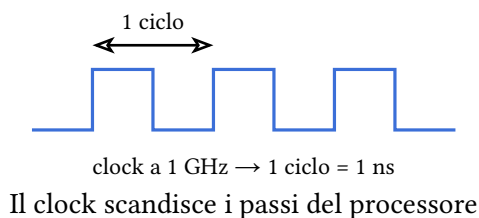
```

inizio:  ADD R0, R0, R0    ← PC
        ADD R1, R1, R1
        SUB R2, R0, R1
        salto inizio

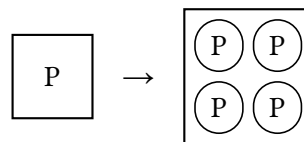
```

Il PC scende di un'istruzione alla volta; il **salto** lo rimette su `inizio`, quindi il programma ricomincia.

A dare il ritmo ai passi del ciclo è il **clock**, un segnale che passa da 0 a 1 e viceversa a intervalli regolari. Ogni passo del processore dura un certo numero di cicli di clock: più cicli al secondo (la **frequenza**, in Hz), più istruzioni al secondo.



- 1 GHz = 10^9 cicli al secondo, quindi un ciclo dura 10^{-9} s = 1 ns.
- Il processore è fatto di **transistor**, minuscoli interruttori. Nel Motorola 68000 (fine anni '70) misuravano 20-40 μm , oggi 2 nm ($1000 \text{ nm} = 1 \mu\text{m}$).
- Transistor più piccoli → nello spazio di un processore ne stanno più di uno. Ogni processore sul chip si chiama **core**, il chip è **multicore**:



Da sapere — legge di Moore (aggiunto, non spiegato in aula)

Il numero di transistor che si riescono a mettere su un chip raddoppia circa ogni due anni. È per questo che si passa dai μm ai nm e da un processore a più processori sullo stesso chip.

1.2 Astrazione

Un calcolatore si studia a **livelli di astrazione**: ognuno usa quello sotto senza doverne conoscere i dettagli.

Applicazioni
Sistema operativo — processi, memoria, file system
Architettura — set di istruzioni (ARMv7)
Microarchitettura — il processore
Logica — componenti: AND, OR, NOT, «memoria»
Circuiti digitali — segnali «1» e «0»: FA, MUX
Circuiti analogici
Dispositivi
Fisica — questi ultimi 3 non li facciamo

Il corso parte dal basso (circuiti digitali, logica) e risale fino al sistema operativo. FA e MUX sono due circuiti che vedremo più avanti: il **full adder** somma una cifra, il **multiplexer** sceglie fra due segnali.

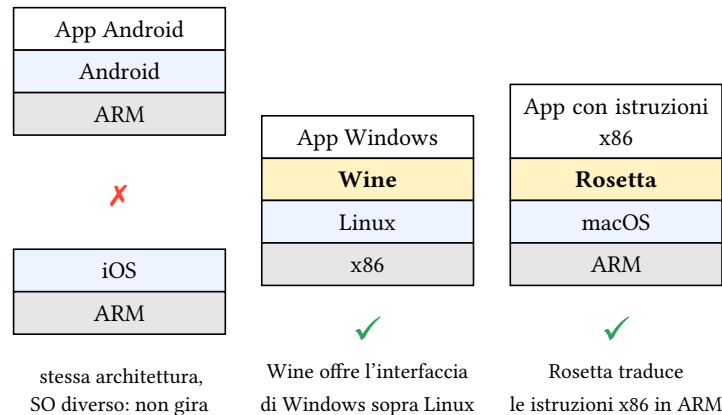
Come si parlano due livelli vicini?

Applicazione — livello i+1
interfaccia: chiamate al sistema operativo
Sistema operativo — livello i
interfaccia: istruzioni in linguaggio macchina
Architettura — livello i-1

Fra due livelli c'è un'**interfaccia**: l'insieme delle funzionalità che il livello i-1 mette a disposizione del livello i.

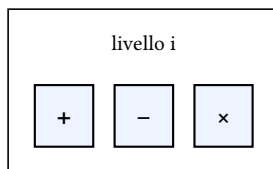
Il livello i usa solo l'interfaccia, non sa com'è fatto sotto.

Conseguenza: un programma gira solo dove trova l'interfaccia per cui è stato scritto. Un'app Android usa le chiamate di sistema di Android: su iOS non ci sono, anche se il processore (ARM) è lo stesso. Al contrario, basta che qualcuno offra l'interfaccia giusta, anche «finta», e il programma gira:

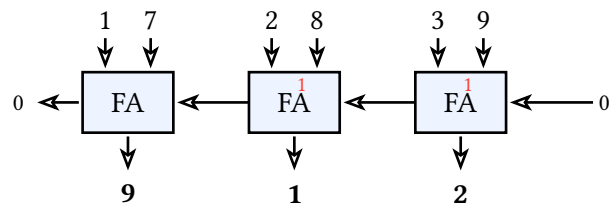


Per costruire sistemi così complessi si usano tre principi:

- **Gerarchia:** il sistema è diviso in livelli (sopra).
- **Modularità:** ogni livello è fatto di moduli con un compito preciso.
- **Regolarità:** lo stesso modulo si riusa tante volte.



Modularità: un livello fatto di moduli

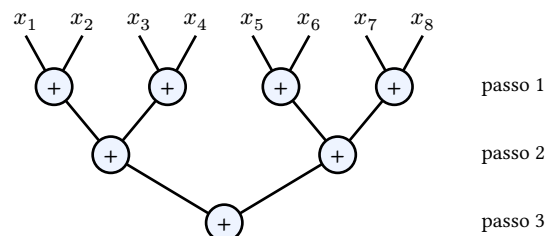


Regolarità: $123 + 789 = 912$ con tre FA uguali in cascata. Ogni FA somma una cifra e passa il riporto a sinistra

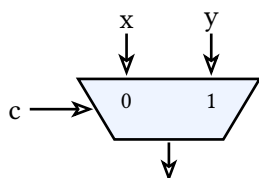
Stesso modulo **+**, due modi di sommare 8 numeri:

```
sum = 0;
for (i = 0; i < n; i++)
    sum = sum + x[i];
```

7 somme **una dopo l'altra**: ognuna aspetta il risultato della precedente, quindi 7 passi.



Ad albero: le somme dello stesso livello sono indipendenti, quindi si possono fare insieme (con 4 sommatore): le stesse 7 somme in 3 passi



MUX: il segnale che entra di lato sceglie quale ingresso passa in uscita, x (se c vale 0) o y (se c vale 1).

1.3 Numeri binari

I circuiti digitali lavorano in **binario**: ogni segnale vale 0 o 1.

In un sistema posizionale **conta l'ordine delle cifre**: la stessa cifra vale di più quanto più è a sinistra. Ogni posizione ha un peso.

decimale	$10^3 = 1000$	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
binario	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

Esempio: $13_{10} \neq 31_{10}$ e $10_2 \neq 01_2$. Stesse cifre, ordine diverso, numero diverso.

Per passare **da binario a decimale** sommo i pesi delle posizioni dove c'è 1: $1100_2 = 8 + 4 = 12$.

Per passare **da decimale a binario** divido per 2 finché arrivo a 1, poi leggo i resti **dal basso verso l'alto**. Il resto della divisione per 2 dice se il numero è pari o dispari, cioè l'ultima cifra binaria; dividendo, «tolgo» quella cifra e passo alla successiva. Per questo le cifre escono da destra a sinistra.

n	: 2	resto
12	6	0
6	3	0
3	1	1
1		1 ↑

$$12 = 1100_2$$

n	: 2	resto
7	3	1
3	1	1
1		1 ↑

$$7 = 111_2$$

Somma e prodotto in binario si fanno come in decimale, ma $1 + 1 = 10_2$: scrivo 0 e riporto 1.

$$\begin{array}{r} 8421 \\ 1010 \\ + 0100 \\ \hline 1110 \end{array}$$

$10 + 4 = 14$ (pesi in grigio)

$$\begin{array}{r} 111 \\ 0111 \\ + 0001 \\ \hline 1000 \end{array}$$

$7 + 1 = 8$ (riporti in grigio)

Il prodotto si fa in colonna come in decimale. In binario ogni riga è molto semplice: la cifra del moltiplicatore è 0 o 1, quindi la riga è tutta zeri oppure il numero stesso. Come in decimale, ogni riga si sposta di un posto a sinistra (il $\cdot$ segna il posto lasciato vuoto).

$$\begin{array}{r} 12 \\ \times 34 \\ \hline 48 \\ 36 \cdot \\ \hline 408 \end{array}$$

in decimale

$$\begin{array}{r} 1010 \\ \times 0100 \\ \hline 0000 \\ 0000 \cdot \\ 1010 \cdot\cdot \\ 0000 \cdot\cdot\cdot \\ \hline 0101000 \end{array}$$

$$10 \times 4 = 40 = 32 + 8$$

Per i **numeri negativi** il modo più semplice è **modulo e segno**: il primo bit è il **segno** (0 = +, 1 = -), gli altri sono il **valore assoluto**.

1	1	1	0	0
segno	valore assoluto = 12			

-12 in modulo e segno

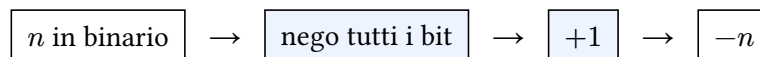
Per sommare A e B bisogna prima guardare i segni:

Caso	Risultato
$S_A = S_B$	segno S_A , valore $V_A + V_B$
$S_A \neq S_B$ e $V_A > V_B$	segno S_A , valore $V_A - V_B$
altrimenti	segno S_B , valore $V_B - V_A$

Esempi: $+12 + 12 = +24$, $-12 + (-12) = -24$, $-12 + 7 = -5$ (segni diversi: faccio $12 - 7$ e tengo il segno del più grande).

Servono confronti e sottrazioni: il circuito che somma diventa complicato. Il complemento a 2 risolve questo problema.

Si usa quindi il **complemento a 2**: i positivi si scrivono come sempre, e per passare da n a $-n$:



Vale anche al contrario: rifacendo «nego + 1» su $-n$ torno a n . Così, se un risultato comincia con 1 (è negativo), capisco quanto vale.

Il vantaggio: **la somma si fa come una somma normale**, senza guardare i segni. Perché funzioni si capisce più sotto, leggendo i pesi dei bit.

Esempio: $-12 + 7$ su 5 bit.

12	01100	10100	somma	11011
nego	10011	+ 00111	nego	00100
+1	10100	11011	+1	00101
-12		-12 + 7	inizia per 1, è negativo:	
			vale -5 ✓	

Esempio: $-5 + (-3)$ su 6 bit.

				11111		
				111011		somma 111000
5	000101	3	000011	+ 111101		nego 000111
nego	<u>111010</u>	nego	<u>111100</u>	<u>111000</u>		+1 001000
+1	111011	+1	111101			
	-5		-3	esce un riporto 1		vale 8, quindi
				oltre i 6 bit: si butta		il risultato è -8 ✓

Nell'esempio $-5 + (-3)$ esce un riporto oltre i 6 bit e il risultato è giusto lo stesso: **il riporto in uscita non è un errore**. L'errore vero è l'overflow, qui sotto.

Quanti numeri ci stanno in n bit? Ogni bit raddoppia le combinazioni: $2 \cdot 2 \cdot 2 \dots = 2^n$. Con 8 bit sono $2^8 = 256$ combinazioni, e il bit in posizione i pesa 2^i .

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

binario	valore
0000 0000	0
0000 0001	1
0000 0010	2
0000 0011	3
0000 0100	4
⋮	⋮
1111 1111	255

Le 256 combinazioni si possono usare in due modi:

con n bit	intervallo	8 bit
interi senza segno (solo +)	$0 \dots 2^n - 1$	$0 \dots 255$
relativi (+ e -) in complemento a 2	$-2^{n-1} \dots + 2^{n-1} - 1$	$-128 \dots + 127$

In complemento a 2 metà delle combinazioni va ai negativi e metà a zero e positivi: per questo c'è un negativo in più (-128) e i positivi si fermano a $+127$.

Ecco perché il complemento a 2 funziona. Un numero in complemento a 2 si legge così: **il bit più a sinistra pesa -2^{n-1}** , gli altri pesano $+2^i$ come sempre.

-128	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$$1111\ 1101 = -128 + 64 + 32 + 16 + 8 + 4 + 1 = -3$$

valore	8 bit
3	0000 0011
2	0000 0010
1	0000 0001
0	0000 0000
-1	1111 1111
-2	1111 1110
-3	1111 1101
⋮	⋮
-128	1000 0000

- Tutti i negativi cominciano per 1, zero e positivi per 0.
- -1 è tutti 1: $0000\ 0001 \rightarrow$ nego $1111\ 1110 \rightarrow +1 \rightarrow 1111\ 1111$.
- -128 non ha l'opposto: facendo «nego + 1» torna se stesso, perché $+128$ non ci sta in 8 bit.

$$\begin{array}{r} -128\ 1000\ 0000 \\ \text{nego}\ 0111\ 1111 \\ \hline +1\ 1000\ 0000 \end{array}$$

In **modulo e segno** l'intervallo è $-(2^{n-1} - 1) \dots + (2^{n-1} - 1)$ e lo zero ha **due** rappresentazioni, $+0 = 0000\ 0000$ e $-0 = 1000\ 0000$. Anche un semplice `if (x == 0)` deve controllarle tutte e due: un altro motivo per cui si usa il complemento a 2.

Sommando due numeri di n bit il risultato può aver bisogno di $n + 1$ bit, ma il circuito ne ha solo n . Quando il risultato esce dall'intervallo rappresentabile si ha **overflow** (traboccamento).

Esempi su 3 bit, dove il complemento a 2 va da -4 a $+3$:

-4	-3	-2	-1	0	1	2	3
100	101	110	111	000	001	010	011

$$\begin{array}{r} 1 \\ 010 \\ + 010 \\ \hline 100 \end{array}$$

$2 + 2 = 4$, ma 100 vale -4 ✗

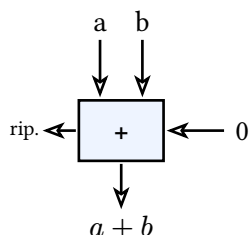
$$\begin{array}{r} 101 \\ + 110 \\ \hline (1)011 \end{array}$$

$-3 + (-2) = -5$, ma 011 vale $+3$ ✗

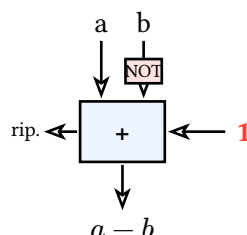
Regola: operandi con lo **stesso segno** e risultato con **segno diverso** $\Rightarrow$ **overflow**.

Con segni diversi l'overflow non può mai capitare: il risultato sta sempre fra i due operandi.

Il complemento a 2 permette di fare anche la **sottrazione con lo stesso sommatore**: $a - b = a + (-b)$, e $-b$ = nego b e sommo 1. Il $+1$ entra dal riporto in ingresso del bit più a destra, che per la somma normale vale 0.



Somma: riporto in ingresso 0



Sottrazione: NOT su b, riporto in ingresso 1

1.4 Basi, numeri reali e caratteri

Un sistema posizionale in **base B** usa le cifre da 0 a $B - 1$, e la cifra in posizione i pesa B^i .

base	cifre	nota
$B = 10$	0, 1, ..., 9	
$B = 2$	0, 1	
$B = 8$	0, 1, ..., 7	una cifra = 3 bit
$B = 16$	0, ..., 9, A, B, C, D, E, F	una cifra = 4 bit (nibble); A = 10, ..., F = 15

In base 16 i pesi sono $16^2 = 256$, $16^1 = 16$, $16^0 = 1$. L'esadecimale si usa perché da binario si passa **senza conti**: $16 = 2^4$, quindi una cifra esadecimale corrisponde esattamente a 4 bit. Divido i bit in gruppi di 4 partendo da destra e scrivo ogni gruppo come una cifra. Il prefisso **0x** indica un numero esadecimale.

$$\begin{array}{cccc} & 6 & & D \\ \hline 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ \hline 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \end{array} \quad 0110 \ 1101 = 0x6D = 64 + 32 + 8 + 4 + 1 = 109$$

Esempio: -3 su 16 bit.

$$\begin{array}{r} 3 \quad 0000 \ 0000 \ 0000 \ 0011 \\ \text{nego} \quad 1111 \ 1111 \ 1111 \ 1100 \\ \hline +1 \quad 1111 \ 1111 \ 1111 \ 1101 \end{array}$$

ogni gruppo 1111 è una F, 1101 è D:
 $-3 = 0xFFFFD$

Per i **numeri con la virgola** la notazione posizionale continua a destra della virgola con pesi negativi:

$$123,45 = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

Tenendo la virgola in una posizione fissa, per sommare $123,45 + 17,9$ bisogna prima allinearla (017,90). Si usa invece la **virgola mobile (floating point)**: cifre ed esponente separati, come nella notazione **mmmEn** = $mmm \times 10^n$.

$$\begin{aligned} 12345E-2 &= 123,45 \quad (\text{la virgola si sposta di 2 a sinistra}) \\ 123E2 &= 12300 \end{aligned}$$

Il numero si scrive come $\pm 0,\text{mantissa} \times 10^E$ e si salvano tre campi, in 32 o 64 bit:

S	E	mantissa
segno	esponente	valore 0,xxxx

$$123,45 = +0,12345 \times 10^3$$

$S = 0, E = 3, \text{ mantissa} = 12345$

L'esponente può essere negativo: con 8 bit ha 256 valori, da -128 a $+127$. Così con pochi bit si scrivono numeri molto grandi e molto piccoli. Si chiama virgola **mobile** proprio perché cambiando l'esponente la virgola si sposta.

Anche i **caratteri** sono numeri. Il codice **ASCII** assegna un numero a ogni carattere, con 7 o 8 bit per carattere (cioè 128 o 256 caratteri); deve rappresentare:

- le 26 lettere, maiuscole e minuscole (e quelle accentate);
- le 10 cifre e la punteggiatura;
- i caratteri speciali: CR (a capo), LF (nuova riga), TAB, DEL, ...

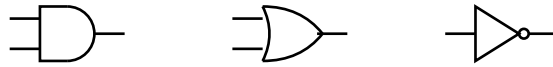
carattere	A	B	a	c	m
codice	65	66	97	99	109

Le lettere hanno codici in ordine alfabetico, quindi confrontare due stringhe (in C `char *s1, *s2`) vuol dire confrontare i codici carattere per carattere: `"ciao" < "mondo"` perché `c = 99 < m = 109`.

2 Reti logiche

2.1 Porte logiche e tabelle di verità

I circuiti digitali lavorano su $\{0, 1\}$ con tre operazioni di base, le **porte logiche**. Gli ingressi entrano da sinistra, l'uscita esce a destra.



AND			OR			NOT	
x	y	z	x	y	z	x	z
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

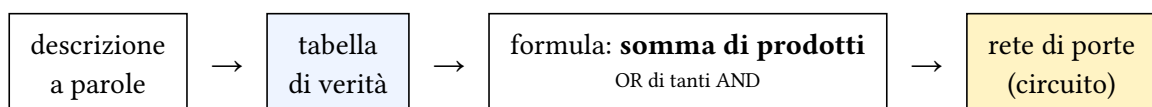
AND vale 1 solo se **tutti** gli ingressi valgono 1; OR vale 1 se **almeno uno** vale 1; NOT inverte. Nelle formule si usa una notazione più corta, come nell'algebra:

	AND	OR	NOT
si scrive	$x \cdot y$ oppure xy (prodotto)	$x + y$ (somma)	$\bar{x}$

Somiglia davvero ad aritmetica: su 0 e 1, AND è il prodotto. OR è la somma, tranne $1 + 1$ che fa 1. Alcune proprietà:

associativa	$(x \text{ AND } y) \text{ AND } z \equiv x \text{ AND } (y \text{ AND } z)$	
commutativa	$x \text{ AND } y \equiv y \text{ AND } x$	
elemento neutro	$x \text{ AND } 1 \equiv x$	$x \text{ OR } 0 \equiv x$
elemento assorbente	$x \text{ AND } 0 \equiv 0$	$x \text{ OR } 1 \equiv 1$

Una funzione logica si può descrivere in tre modi, e da uno si passa all'altro:



Esempio: il **MUX**. A parole: *scegli fra due ingressi x e y a seconda di un ingresso di controllo c .*

```

if (c == 0) z = x;
else      z = y;
  
```

Dalla descrizione si ricava la tabella di verità. Poi, per ogni riga dove $z = 1$, scrivo un prodotto (AND) che vale 1 **solo** in quella riga: la variabile così com'è se vale 1, negata ($\bar{x}$) se vale 0.

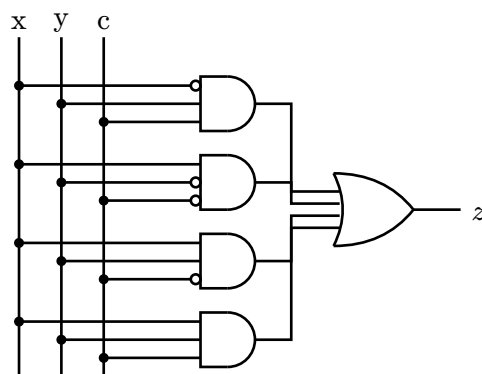
x	y	c	z	prodotto
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\bar{x}yc$
1	0	0	1	$x\bar{y}\bar{c}$
1	0	1	0	
1	1	0	1	$xy\bar{c}$
1	1	1	1	xyz

Le righe con $z = 1$ sono unite da un OR. Funziona perché ogni prodotto vale 1 solo nella sua riga, e l'OR vale 1 appena uno dei prodotti vale 1: la formula vale 1 esattamente nelle righe scelte.

$$z = \bar{x}yc + x\bar{y}\bar{c} + xy\bar{c} + xyz$$

Per esempio $\bar{x}yc = \text{NOT}(x) \text{ AND } y \text{ AND } c$: vale 1 solo per $x = 0, y = 1, c = 1$.

Ogni prodotto diventa una porta AND a tre ingressi (il pallino sull'ingresso è un NOT), e le quattro uscite entrano in una OR:



Il MUX come rete di porte

... che si disegna col simbolo

